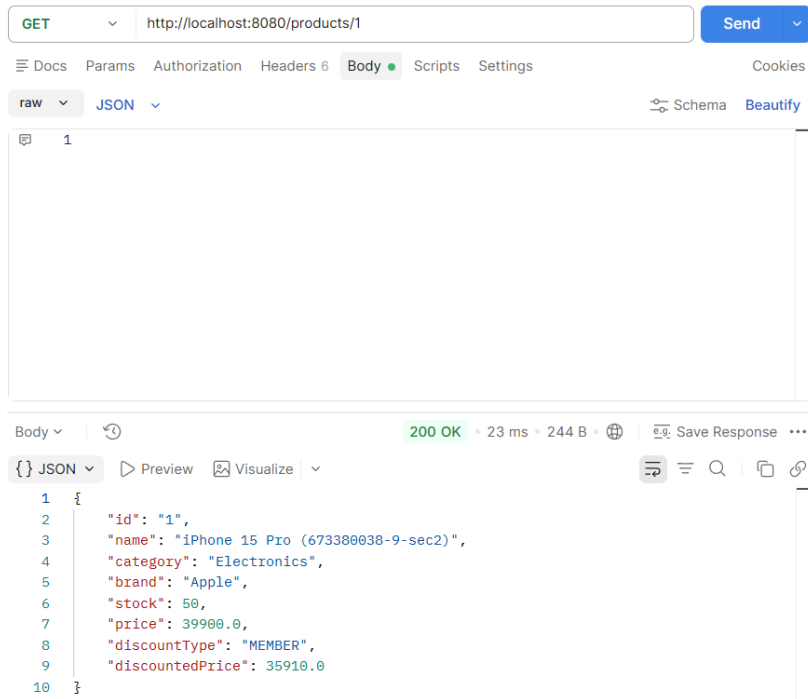


Endpoint Lab10



GET http://localhost:8080/products/1

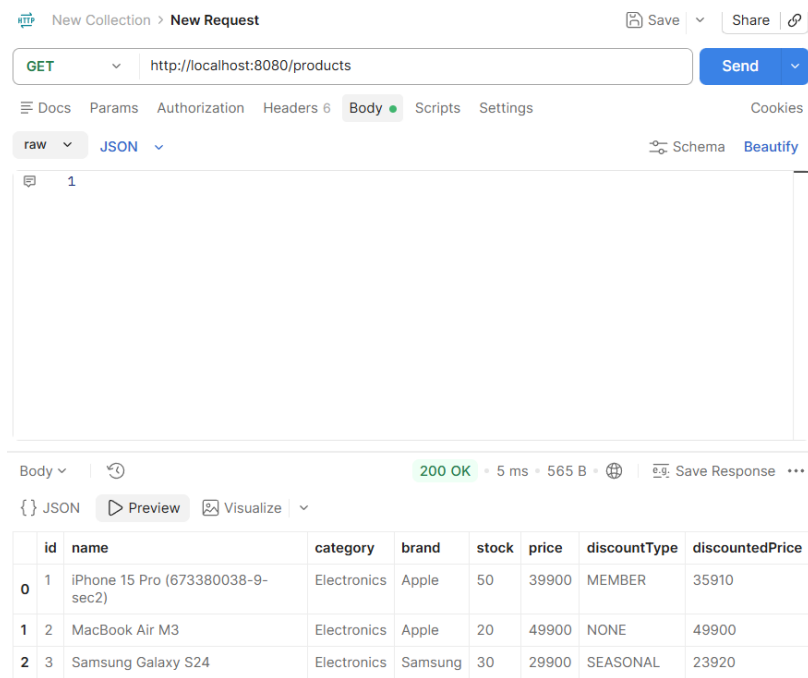
200 OK • 23 ms • 244 B

```

{
  "id": "1",
  "name": "iPhone 15 Pro (673380038-9-sec2)",
  "category": "Electronics",
  "brand": "Apple",
  "stock": 50,
  "price": 39900.0,
  "discountType": "MEMBER",
  "discountedPrice": 35910.0
}
```

* GET /products/{id}

* คำน Mono<Product> — ค้นหา Product 1 รายการ



GET http://localhost:8080/products

200 OK • 5 ms • 565 B

```

[
  {
    "id": "1",
    "name": "iPhone 15 Pro (673380038-9-sec2)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900.0,
    "discountType": "MEMBER",
    "discountedPrice": 35910.0
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900.0,
    "discountType": "NONE",
    "discountedPrice": 49900.0
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900.0,
    "discountType": "SEASONAL",
    "discountedPrice": 23920.0
  }
]
```

	id	name	category	brand	stock	price	discountType	discountedPrice
0	1	iPhone 15 Pro (673380038-9-sec2)	Electronics	Apple	50	39900	MEMBER	35910
1	2	MacBook Air M3	Electronics	Apple	20	49900	NONE	49900
2	3	Samsung Galaxy S24	Electronics	Samsung	30	29900	SEASONAL	23920

* GET /products

* TODO: คำน Flux<Product> ทุกรายการ

New Collection > New Request

POST http://localhost:8080/products Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```

1 {
2   "id": "4",
3   "name": "Test Product",
4   "category": "Test Category",
5   "brand": "BrandX",
6   "stock": 10,
7   "price": 100.0,
8   "discountType": "NONE"
9 }

```

Body 200 OK • 81 ms • 221 B • Save Response

{ } JSON Preview Visualize

id	4
name	Test Product
category	Test Category
brand	BrandX
stock	10
price	100
discountType	NONE
discountedPrice	100

* POST /products

* TODO: รับ Product จาก request body แล้วบันทึก

New Collection > New Request

DELETE http://localhost:8080/products/4 Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

raw JSON Schema Beautify

1

Body 200 OK • 5 ms • 40 B • Save Response

Raw Preview Visualize

* DELETE /products/{id}

* TODO: ลบ Product และคืน Mono<Void>

GET http://localhost:8080/products/category/Electronics Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

raw JSON Schema Beautify

1

Body 200 OK • 7 ms • 565 B Save Response

{ } JSON Preview Visualize

	id	name	category	brand	stock	price	discountType	discountedPrice
0	1	iPhone 15 Pro (673380038-9-sec2)	Electronics	Apple	50	39900	MEMBER	35910
1	2	MacBook Air M3	Electronics	Apple	20	49900	NONE	49900
2	3	Samsung Galaxy S24	Electronics	Samsung	30	29900	SEASONAL	23920

* GET /products/category/{category}

* TODO: คืน Flux<Product> ที่กรองตาม category

New Collection > New Request Save Share

GET http://localhost:8080/products/1/price Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

raw JSON Schema Beautify

1

Body 200 OK • 17 ms • 79 B Save Response

{ } JSON Preview Visualize

35910

* GET /products/{id}/price

* TODO: คืน Mono<Double> ราคาหลังส่วนลด

1. Reactive Programming vs Blocking I/O

หัวข้อ	Blocking I/O (Imperative)	Reactive Programming (Non-blocking)
Model การทำงาน	Thread-per-request (1 Request ต่อ 1 Thread)	Event-Loop / Asynchronous Event-driven
การจัดการ I/O	Thread หยุดรอ (Blocked) จนกว่า I/O หรือ DB จะตอบกลับ	Thread ไม่หยุดรอ (Non-blocking) สามารถสลับไปทำงานอื่นได้ทันที
การรองรับ Concurrency	จำกัดตามจำนวน Thread Pool (สิ้นเปลือง Memory)	รองรับ Concurrent Requests สูงมากด้วย Thread จำนวนน้อย
รูปแบบสินค้า	สินค้าเป็น Object ตรงๆ เช่น Product, List<Product>	สินค้าเป็น Data Stream (Publisher) เช่น Mono<T>, Flux<T>

2. Mono vs Flux

- **Mono<T> (0 หรือ 1 ค่า):**
 - **ใช้กรณี:** ดึงข้อมูลแบบเจาะจงด้วย ID (findById), การบันทึก/แก้ไขข้อมูล (save), หรือการดำเนินการที่ไม่ต้องการสินค้า (Mono<Void>)
 - **เหตุผล:** เพื่อบอกระบบและผู้เรียกใช้งานชัดเจนว่า Operation นี้จะคืนผลลัพธ์เพียงชิ้นเดียวหรือไม่คืนเลย (Empty)
- **Flux<T> (0 ถึง N ค่า):**
 - **ใช้กรณี:** ดึงข้อมูลรายการทั้งหมด (findAll), การค้นหาข้อมูลตามเงื่อนไขที่อาจมีหลายรายการ (findByCategory), หรือการทำ Data/Event Streaming
 - **เหตุผล:** ออกแบบมาเพื่อจัดการลำดับของข้อมูล (Stream of Elements) ที่ทยอยส่งผลลัพธ์มาทีละชิ้น และรองรับการทำ Backpressure (การควบคุมอัตราการรับส่งข้อมูล)

3. การทำงานของ WebClient Method Chain

- **.get() / .post() / .delete():** ระบุ HTTP Method ที่ต้องการส่ง Request ไปยังปลายทาง
- **.uri("/products/{id}", id):** กำหนด Endpoint Path พร้อมทำการแทนค่าตัวแปร (Path Variable / Query Parameter) เข้าไปใน URL อย่างปลอดภัย
- **.retrieve():** สั่ง Execute คำสั่ง HTTP Request ออกไป และเริ่มกระบวนการดักจับ HTTP Response กลับมาประมวลผล
- **.bodyToMono(Product.class) / .bodyToFlux(Product.class):** ทำหน้าที่ แปลง (Deserialize) โครงสร้างข้อมูล JSON จาก Response Body ให้กลายเป็น Java Object ตามประเภทที่กำหนด โดยคืนค่าเป็น Mono<Product> สำหรับ Object เดียว หรือ Flux<Product> สำหรับกลุ่มของ Object

4. โค้ดสำหรับ Repository, Service และ Controller

ProductRepository.java

Java

```
package com.example.lab10.repository;
```

```
import com.example.lab10.model.Product;
```

```
import org.springframework.stereotype.Repository;
```

```
import reactor.core.publisher.Flux;
```

```
import reactor.core.publisher.Mono;
```

```
import java.util.Map;
```

```
import java.util.concurrent.ConcurrentHashMap;
```

```
@Repository
```

```
public class ProductRepository {
```

```
    private final Map<String, Product> store = new ConcurrentHashMap<>();
```

```
    public ProductRepository() {
```

```
store.put("1", new Product("1", "iPhone 15 Pro (673380038-9-sec2)",  
"Electronics", "Apple", 50, 39900.0, "MEMBER"));  
  
store.put("2", new Product("2", "MacBook Air M3", "Electronics", "Apple", 20,  
49900.0, "NONE"));  
  
store.put("3", new Product("3", "Samsung Galaxy S24", "Electronics",  
"Samsung", 30, 29900.0, "SEASONAL"));  
}
```

```
public Mono<Product> findById(String id) {  
    Product product = store.get(id);  
    if (product == null) {  
        return Mono.empty();  
    }  
    return Mono.just(product);  
}
```

```
public Flux<Product> findAll() {  
    return Flux.fromIterable(store.values());  
}
```

```
public Mono<Product> save(Product product) {  
    store.put(product.getId(), product);  
    return Mono.just(product);  
}
```

```
public Mono<Void> deleteById(String id) {  
    store.remove(id);  
    return Mono.empty();  
}
```

```
public Flux<Product> findByCategory(String category) {  
    return findAll().filter(p -> p.getCategory().equalsIgnoreCase(category));  
}
```

```
}  
}
```

ProductService.java

Java

```
package com.example.lab10.service;
```

```
import com.example.lab10.model.Product;
```

```
import com.example.lab10.repository.ProductRepository;
```

```
import org.springframework.stereotype.Service;
```

```
import reactor.core.publisher.Flux;
```

```
import reactor.core.publisher.Mono;
```

```
import java.util.UUID;
```

```
@Service
```

```
public class ProductService {
```

```
    private final ProductRepository repository;
```

```
    public ProductService(ProductRepository repository) {
```

```
        this.repository = repository;
```

```
    }
```

```
    public Mono<Product> getByld(String id) {
```

```
        return repository.findByld(id)
```

```
            .switchIfEmpty(Mono.error(new RuntimeException("Product not found: " +  
id)));
```

```
    }
```

```
    public Flux<Product> getAll() {
```

```

    return repository.findAll();
}

public Mono<Product> save(Product product) {
    if (product.getId() == null) {
        product.setId(UUID.randomUUID().toString());
    }
    return repository.save(product);
}

public Mono<Void> delete(String id) {
    return repository.deleteById(id);
}

public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

public Mono<Double> getDiscountedPrice(String id) {
    return repository.findById(id).map(Product::getDiscountedPrice);
}
}

```

ProductController.java

Java

```

package com.example.lab10.controller;

import com.example.lab10.model.Product;
import com.example.lab10.service.ProductService;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

```

@RestController

@RequestMapping("/products")

public class ProductController {

private final ProductService service;

public ProductController(ProductService service) {

this.service = service;

}

@GetMapping("/{id}")

public Mono<Product> getById(@PathVariable String id) {

return service.getById(id);

}

@GetMapping

public Flux<Product> getAll() {

return service.getAll();

}

@PostMapping

public Mono<Product> save(@RequestBody Product product) {

return service.save(product);

}

@DeleteMapping("/{id}")

public Mono<Void> delete(@PathVariable String id) {

return service.delete(id);

}

```
@GetMapping("/category/{category}")
public Flux<Product> getByCategory(@PathVariable String category) {
    return service.getByCategory(category);
}

@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    return service.getDiscountedPrice(id);
}
}
```